Rules that override everything below:ANTI-VIBE-CODING RULES (apply to everything you build)

A. DESIGN: do not let the model pick the look

1. The design is specified in this prompt. Do not choose trends, palettes, fonts or layouts yourself. Left to itself, an LLM picks the most common choices, and the result looks like every other AI-built site.
2. No gradients of any kind (backgrounds, buttons, text, borders, fades, skeleton shimmer). No purple or blue-purple. No glow, neon, blur or glassmorphism.
3. One accent color. Semantic colors (green, amber, red) only for real status, as a dot plus text. No rainbow icon tiles or colored stat callouts.
4. No card for everything. No bento grids, generic dashboards, stat tiles, feature-card rows, or cards with colored stripes. Organize with whitespace, alignment and 1px dividers.
5. Max two font weights, a fixed type scale, a 4px spacing grid, and at most three corner radii. Avoid defaulting to Inter. Mono type is for technical strings only, never decoration (no "//" prefixes, no terminal styling).
6. No emoji or sparkles in the UI or the copy. Icons come from one library, one size per context, one stroke weight, properly aligned with text.
7. No pulsing or looping animation. No always-on "Active", "Verified", "Live" or "Secure" badges. No cursor-following effects, moving buttons, hover-only functionality, scroll hijacking, lines that follow the scroll, or fade-ins on scroll.
8. Motion only explains a state change (connecting, selecting, sending, done). Respect prefers-reduced-motion.
9. Copy is plain and specific. Banned words: elevate, seamless, next-generation, supercharge, unleash, empower, effortless, revolutionary, magic, cutting-edge, intuitive. No gray subtitle under every heading. No technology credits on screen ("Built with React").
10. Functional screens are tools, not landing pages. No hero sections, testimonials, logo clouds or link-heavy footers unless I ask.
11. One visual system across every screen. If two sections look like they were generated separately, fix them.
12. Mobile is designed on its own (single column, 44px targets, safe-area insets), not shrunken desktop.
13. Every element must have a purpose. If it only decorates, delete it.

B. ENGINEERING: no fake or fragile code

14. Nothing fake. No hardcoded data pretending to be live, no simulated results, no placeholder functions, no buttons that do nothing. If a feature is not built, disable it and say why.

15. Never claim a capability the platform does not have. State limits honestly in the UI.

16. No giant single files or components. Separate concerns into small, typed modules. No "any" types.

17. Handle every failure path: permission denied, offline, empty state, loading state, timeout, bad input, unsupported browser. Never show a blank screen, and never swallow errors silently.

18. Clean up everything: listeners, timers, media streams, object URLs, connections, intervals.

19. Do not push high-frequency data (camera frames, mouse moves, animation ticks) through React state.

20. Use only packages that exist, pinned to stable versions. Do not invent APIs or library functions. If unsure an API exists, say so instead of guessing. Add no dependency without a stated reason.

21. No secrets or API keys in client code. No unneeded backend, auth or database.

22. Validate all external input (pasted codes, files, URLs) before using it.

23. No console errors or warnings. No unused code or dead imports.

C. PROCESS: no overbuilding, no false "done"

24. Build in phases. Complete one phase, run it, fix errors, report, then stop and wait.

25. Do not rewrite or restructure working code unless I ask. Do not add features I did not ask for.

26. Do not say something works unless you have verified it. Every report lists: what works, what was tested, what is unavailable or unverified.

27. When fixing a bug, find the cause. Do not paper over it with a try/catch or a workaround that hides it.

28. Before each report, audit against sections A and B: search the code for gradient, blur, backdrop-filter, pulse animations, emoji, off-palette colors and unused code, and fix what you find.

THE REAL PROMPT:

MASTER ENGINEERING PROMPT — EARTH UNCOVERED

Build a Photorealistic, Browser-Based, 3D Earth Exploration & Geography Game

You are acting as a principal graphics engineer, senior full-stack architect, 3D rendering engineer, geospatial systems engineer, performance engineer, and product designer.

Your responsibility is to engineer a production-oriented browser-based 3D geographic exploration platform called **Earth Uncovered**.

This is NOT a conventional GeoGuessr clone.

It is an ambitious project to build a photorealistic, explorable, geographically accurate representation of the real world, where players can move through actual reconstructed locations, investigate their surroundings, and identify where on Earth they are.

The experience should combine:

- The geographical accuracy of Google Earth.
- The accessibility of a browser game.
- The visual realism of high-quality photogrammetry.
- The exploration mechanics of a first-person game.
- The competitive gameplay of GeoGuessr.
- The streaming efficiency of a modern geographic rendering engine.

Your primary objective is to build the underlying technology correctly, not to create an impressive-looking mockup.

---

1. PRODUCT VISION

Build a browser-based game where a player is randomly placed somewhere on Earth inside a genuinely navigable 3D reconstruction of a real-world location.

The player should be able to:

1. Walk forward, backward, left and right.
2. Rotate the camera freely.
3. Explore streets and nearby surroundings.
4. Observe actual architecture, vegetation, roads, buildings and environmental details.
5. Investigate geographical clues such as signs, road markings, shop names and landmarks.
6. Open a world map.

7. Place a marker to guess their current location.
8. Submit the guess.
9. Receive a score based on the distance between the guess and the actual location.

The long-term objective is global geographic coverage.

However, do not pretend that the entire planet is already reconstructed in photorealistic 3D.

The system must distinguish between:

- Geographic coverage.
- Terrain coverage.
- Building geometry coverage.
- Photorealistic reconstruction coverage.
- Collision and navigation coverage.

A location must only be selected for a playable game round if it meets the required quality and navigation criteria.

Product identity

Name: Earth Uncovered

Core concept: Explore the real world. Discover where you are.

The product must feel like a serious geography platform, not a generic AI-generated gaming website.

---

2. NON-NEGOTIABLE PROJECT REQUIREMENTS

These requirements override convenience and shortcuts.

Hardware compatibility

The primary target is average consumer hardware.

Support:

- Ordinary laptops.
- Integrated graphics.
- Mid-range dedicated GPUs.

- Modern Android smartphones.
- Modern iPhones and supported mobile browsers.

Do not assume that users have powerful gaming GPUs.

Visual priority

Balanced photorealism and performance.

The environment must look genuinely realistic, but rendering quality must adapt to hardware.

Platform

Browser-first, including mobile browsers.

No mandatory desktop application.

Use progressive enhancement for newer browser graphics capabilities.

Performance philosophy

The entire Earth must NEVER be loaded into browser memory.

The client should only load the geographic regions necessary for the player's current location, camera visibility and likely movement direction.

The engine must have:

- Spatial chunking.
- Hierarchical LOD.
- Progressive asset loading.
- GPU-aware resource management.
- Memory budgets.
- Asset compression.
- Network-aware streaming.
- Intelligent cache eviction.

Visual authenticity

Do not substitute actual-world reconstruction with:

- Static panorama backgrounds.
- Fake 3D CSS effects.

- Stock photographs.
- Random AI-generated city backgrounds.
- Flat satellite images pretending to be 3D.
- Non-navigable screenshots.

A 360-degree panorama alone does not qualify as a 3D world.

The first milestone must demonstrate real 3D scene navigation.

---

3. MANDATORY RESEARCH BEFORE IMPLEMENTATION

Before selecting dependencies or implementing the rendering engine, research current documentation and evaluate the following technologies.

Rendering engines

- Three.js.
- PlayCanvas Engine.
- CesiumJS.
- WebGPU.
- WebGL2.

3D reconstruction

- 3D Gaussian Splatting.
- Photogrammetry.
- COLMAP.
- Nerfstudio.
- Open-source Gaussian Splat renderers.
- PlayCanvas SuperSplat.

Geographic systems

- OGC 3D Tiles.
- OpenStreetMap.
- PostGIS.
- H3.
- Quadtree-based spatial indexing.
- Digital elevation models.

Optimization

- KTX2 / Basis Universal.
- Meshopt.
- Draco.
- Gaussian Splat compression.
- Web Workers.
- OffscreenCanvas where supported.

Research current versions, browser compatibility, licensing, performance characteristics and integration complexity.

Use official documentation and maintained repositories.

Do not choose a technology simply because it is popular.

Architecture selection requirement

Compare at least these approaches:

A. Three.js + custom geographic streaming + Gaussian Splat renderer.

B. PlayCanvas + SuperSplat-based rendering + geographic streaming.

C. CesiumJS + 3D Tiles + custom photorealistic street-level renderer.

Select the most practical architecture for this project.

You may combine technologies if necessary, but explain why.

Do not introduce multiple rendering engines without a clear technical reason.

Do not invent API methods or assume libraries support functionality without verifying their documentation.

Write your findings in:

docs/architecture/TECHNOLOGY_DECISIONS.md

Include:

- Selected technology.
- Alternatives considered.
- Reasons for selection.

- Known limitations.
- Licensing considerations.
- Browser compatibility.
- Performance implications.

Then proceed with implementation.

---

4. CORE WORLD REPRESENTATION

The world must use a hybrid representation.

Do not attempt to represent everything using one rendering technique.

Layer A: Photorealistic reconstruction

Use 3D Gaussian Splatting for captured environments.

The objective is to reproduce:

- Real building appearance.
- Windows and doors.
- Roads and pavements.
- Trees and vegetation.
- Street furniture.
- Surface details.
- Shadows and visual texture.

Support streaming-compatible representations where available.

Evaluate SOG or other current compressed Gaussian representations.

The renderer must support loading and unloading individual scene chunks.

Layer B: Physical geometry

Use conventional 3D geometry for:

- Collision surfaces.

- Walkable roads.
- Pavements.
- Building boundaries.
- Walls.
- Terrain.
- Interactive objects.

Do not assume Gaussian splats automatically provide accurate collision geometry.

Generate or import collision proxies separately.

Layer C: Geographic structure

Maintain a geographic data layer containing:

- Latitude.
- Longitude.
- Elevation.
- Roads.
- Intersections.
- Building footprints.
- Landmarks.
- Terrain.
- Geographic boundaries.

Use WGS84 for global geographic coordinates.

Use ECEF where appropriate for global positioning.

Use local East-North-Up coordinates for local scene rendering and physics.

Avoid floating-point precision problems caused by rendering directly in huge global coordinates.

---

5. WORLD STREAMING ENGINE

THIS IS ONE OF THE MOST IMPORTANT COMPONENTS.

Build a dedicated streaming subsystem.

The browser must not load an entire city, country or planet at once.

Geographic chunking

Divide the environment into hierarchical geographic chunks.

Use:

- Quadtree or hierarchical tile structures for rendering.
- H3 or another geographic indexing system for location organization and coverage management.

Do not use H3 as the only rendering hierarchy.

Each chunk should have:

- Geographic bounds.
- Parent identifier.
- Child identifiers.
- LOD level.
- Asset manifest.
- Bounding volume.
- Asset version.
- Compression metadata.
- Quality metadata.

Dynamic streaming

Implement this sequence:

1. Detect the player's geographic position.
2. Determine the camera frustum.
3. Identify visible chunks.
4. Identify nearby chunks.
5. Prioritize required assets.
6. Check local cache.
7. Download missing assets.
8. Decode assets asynchronously.
9. Upload necessary resources to the GPU.
10. Replace low-detail assets progressively.
11. Remove obsolete resources.
12. Maintain a bounded cache.

Asset priorities

P0: Current visible street and nearby geometry.

P1: Immediate surrounding buildings and environment.

P2: Nearby areas in the likely movement direction.

P3: Distant environment.

P4: Background prefetch.

Requests must be cancellable when the player moves away or changes direction.

LOD

Implement screen-space-error-based or equivalent adaptive LOD.

Near objects should have higher visual detail.

Far objects should use simplified representations.

Do not load maximum-resolution textures for distant objects.

Cache management

Implement:

- Memory-aware cache.
- LRU or priority-based eviction.
- GPU resource cleanup.
- Texture lifecycle management.
- Request deduplication.
- Asset versioning.
- Persistent caching where permitted.

Use a bounded cache, not unlimited browser storage.

The game must remain usable if network connectivity becomes temporarily unstable.

---

6. PHOTOREALISTIC RENDERING ENGINE

Photorealism is a core product requirement.

The environment should not look like a low-quality procedural game.

Prioritize:

- Accurate real-world textures.
- Natural lighting.
- Realistic material appearance.
- Detailed vegetation.
- Realistic building geometry.
- Appropriate environmental shadows.
- Accurate terrain.
- Natural visual transitions.
- High-quality nearby reconstruction.

Rendering backend

Implement capability detection.

Preferred:

- WebGPU when supported and stable for required features.

Fallback:

- WebGL2.

Do not allow the game to crash simply because WebGPU is unavailable.

Provide a common renderer interface so that gameplay and world systems do not depend directly on one rendering backend.

Performance optimizations

Implement:

- Frustum culling.
- Appropriate occlusion culling.
- Geometry LOD.
- Texture LOD.
- Compressed textures.
- Compressed geometry.
- Adaptive resolution.
- GPU resource budgets.
- Background decoding.
- Efficient render loops.
- Reduced rendering when the game tab is inactive.

Avoid excessive:

- Real-time global illumination.
- Screen-space reflections.
- High-resolution dynamic shadows.
- Expensive post-processing.
- Unnecessary particle systems.

Use baked lighting where appropriate.

Do not sacrifice all visual realism for simplistic low-poly graphics.

---

7. FIRST-PERSON MOVEMENT AND PHYSICS

Build a proper first-person controller.

Requirements:

Desktop:

- WASD movement.
- Mouse look.
- Pointer lock where available.
- Sprint.
- Configurable sensitivity.

Mobile:

- Virtual joystick.
- Touch camera control.
- Adjustable sensitivity.
- Landscape support.
- Accessible controls.
- Reduced-effects option.

Movement must use collision geometry.

Players must not:

- Walk through buildings.
- Fall through terrain.
- Move through solid obstacles.
- Float above roads.
- Experience major camera jitter.

Implement:

- Ground detection.
- Collision handling.
- Camera height.
- Movement smoothing.
- Frame-rate-independent movement.
- Sensible movement speed.

Do not overengineer physics.

A lightweight capsule-based controller is sufficient for the initial prototype.

---

8. GEOGRAPHIC DATA AND ASSET INGESTION

Build an independent asset ingestion pipeline.

Potential sources include:

- OpenStreetMap.
- Copernicus DEM.
- Licensed geographic datasets.
- Licensed street-level imagery.
- Our own photographs.
- Our own 360-degree captures.
- Our own video datasets.
- Licensed photogrammetry assets.

Respect all source licenses and API restrictions.

Never scrape restricted imagery or bypass provider access controls.

Do not assume that publicly viewable imagery is freely redistributable.

Reconstruction pipeline

Create a processing pipeline:

CAPTURE

- Images.
- Video.

- GPS coordinates.
- Camera metadata.
- Capture timestamp.

RECONSTRUCTION

- Camera calibration.
- Structure from Motion.
- Photogrammetry.
- Gaussian Splat generation.
- Mesh generation.
- Collision geometry generation.

OPTIMIZATION

- Artifact removal.
- Texture compression.
- Geometry optimization.
- LOD generation.
- Splat compression.
- Chunk packaging.

PUBLISHING

- Asset manifests.
- Object storage.
- CDN.
- Geographic index.
- Coverage metadata.

Reconstruction must happen on processing infrastructure, not inside the player's browser.

The browser should primarily download, decode and render optimized assets.

---

9. COVERAGE MANAGEMENT

Do not claim global photorealistic coverage before it exists.

Build a coverage database.

Each location should track:

- Geographic coverage.
- Terrain coverage.
- Road coverage.
- Building coverage.
- Photorealistic 3D coverage.

- Collision coverage.
- Capture date.
- Asset version.
- Data source.
- License.
- Visual quality score.
- Navigation quality score.

A location must not become playable until it passes validation.

Implement a coverage eligibility service.

The game should select only locations with sufficient:

- Visual completeness.
- Navigable geometry.
- Spawn-point quality.
- Environmental detail.
- Geographic accuracy.

If a location has incomplete 3D reconstruction, it must not be silently presented as fully reconstructed.

---

10. LIVING ENVIRONMENT SIMULATION

The world should eventually feel alive.

Important distinction:

The initial product will use simulated environmental activity, not a live surveillance feed or a representation of real-time street activity.

Build the simulation system as an independent subsystem.

Future capabilities:

- Animated pedestrians.
- Moving vehicles.
- Traffic lights.
- Wind-driven vegetation.
- Changing weather.
- Dynamic day/night.
- Environmental audio.

- Moving clouds.
- Seasonal environmental changes.

Do not implement expensive AI agents for every person or vehicle.

Use lightweight deterministic simulation and instancing where possible.

Only simulate objects that are relevant to the visible environment.

For the initial prototype, environmental animation is optional. Prioritize actual 3D navigation and performance first.

---

11. GAMEPLAY SYSTEM

After the 3D prototype and streaming engine work, implement the game.

Classic mode

- Random playable geographic location.
- Limited exploration time.
- First-person movement.
- World map.
- Guess submission.
- Distance calculation.
- Final score.

Explorer mode

- Unlimited exploration.
- No strict timer.
- Educational geographic experience.

No Movement mode

- Camera rotation allowed.
- No walking.
- Traditional guessing mechanics.

Country Challenge

- Locations restricted to a selected country.

Multiplayer Duel

- Two players.
- Same location.
- Synchronized round state.
- Independent exploration.
- Simultaneous guess submission.
- Server-authoritative scoring.

Scoring

Use geographic distance, preferably calculated using a reliable geodesic method.

An initial scoring function can be:

$S = 5000 \times \exp(-d/k)$

Where:

- d = distance between guess and actual location in kilometres.
- k = configurable difficulty constant.

The backend must calculate the final score.

Never trust client-submitted scores.

Location fairness

Prevent repeated locations where possible.

Avoid locations with:

- Poor visibility.
- Missing reconstruction.
- Incorrect GPS positioning.
- Insufficient clues.
- Unusable navigation geometry.

Store the actual location securely on the server until the round ends.

Do not expose answer coordinates through public APIs or browser metadata before submission.

Understand that complete secrecy is impossible when visual clues themselves reveal the answer.

12. FRONTEND AND USER EXPERIENCE

The interface must feel professionally designed.

Do not make it look AI-generated.

STRICT DESIGN REQUIREMENTS:

- No generic AI dashboard aesthetic.
- No unnecessary gradients.
- No excessive glassmorphism.
- No random glowing borders.
- No giant rounded cards everywhere.
- No meaningless animations.
- No decorative charts.
- No oversized headings.
- No excessive empty space.
- No unnecessary icons.
- No generic purple-blue AI branding.

Use a restrained, sophisticated visual system.

Recommended direction:

- Dark neutral interface for the game.
- Deep charcoal backgrounds.
- Subtle borders.
- Neutral typography.
- Clear information hierarchy.
- Minimal controls.
- Precise spacing.
- Small, purposeful animations.
- Consistent iconography.

The 3D world must be the visual focus.

The game interface should stay out of the player's way.

Main screens

1. Landing page.

2. Game mode selection.
3. Game loading screen.
4. 3D gameplay interface.
5. Guess map.
6. Round results.
7. Multiplayer lobby.
8. Profile.
9. Leaderboard.
10. Settings.

Gameplay layout

The gameplay screen should prioritize the 3D environment.

Controls:

- Minimal movement instructions.
- Small timer.
- Compact settings.
- Expandable guessing map.
- Guess submission control.

On mobile:

- Touch-first controls.
- Responsive guessing map.
- Minimal overlays.
- Proper safe-area handling.
- Landscape gameplay support.

The interface must be usable on small screens.

---

13. BACKEND ARCHITECTURE

Use a modular Node.js + TypeScript backend.

Recommended components:

AUTH SERVICE

- Authentication.
- User profiles.
- Session management.

WORLD SERVICE

- Geographic chunk metadata.
- Asset manifests.
- Coverage information.
- Geographic queries.

LOCATION SERVICE

- Eligible game locations.
- Random location selection.
- Location quality validation.

GAME SERVICE

- Game sessions.
- Rounds.
- Timers.
- Guess submissions.

SCORING SERVICE

- Geodesic distance.
- Score calculations.
- Result validation.

MULTIPLAYER SERVICE

- Room management.
- Matchmaking.
- Realtime events.

ASSET SERVICE

- Asset manifests.
- Version management.
- Signed URLs where necessary.

Database

Use PostgreSQL + PostGIS.

Suggested entities:

- users
- locations
- world_chunks
- world_assets
- captures
- coverage

- game_sessions
- game_rounds
- guesses
- scores
- multiplayer_rooms

Use Redis for:

- Active game sessions.
- Multiplayer state.
- Matchmaking.
- Temporary caching.
- Rate limiting.

Use object storage for:

- Gaussian Splats.
- Textures.
- Meshes.
- Terrain assets.
- Chunk manifests.

Do not store large 3D binary assets directly in PostgreSQL.

---

14. PERFORMANCE ENGINEERING

Performance is a product requirement, not a final optimization task.

Initial targets:

AVERAGE LAPTOP

- Target 45–60 FPS.
- Minimum acceptable 30 FPS.

AVERAGE SMARTPHONE

- Target 30 FPS.
- Minimum acceptable 24 FPS during demanding scenes.

These are targets to validate, not guaranteed results.

Implement:

1. Device capability detection.
2. Adaptive rendering resolution.
3. Dynamic quality settings.
4. Texture and geometry budgets.
5. GPU resource cleanup.
6. Background asset decoding.
7. Request cancellation.
8. Network-aware streaming.
9. LOD management.
10. Visibility-based loading.
11. Mobile thermal awareness.
12. Memory-aware asset eviction.

Never assume GPU memory availability can be measured precisely across all browsers.

Use observed performance and conservative resource budgets.

Provide three profiles:

PERFORMANCE

- Smaller streaming radius.
- Reduced effects.
- Lower resolution.
- Lower splat density.

BALANCED

- Default profile.
- Adaptive resolution.
- Medium-to-high nearby detail.
- Moderate effects.

QUALITY

- Higher detail.
- Larger streaming radius.
- Better lighting.
- More detailed reconstruction.

The system should be able to downgrade quality automatically when frame times become unstable.

---

15. PROJECT STRUCTURE

Use a clean monorepo architecture.

Recommended structure:

earth-uncovered/

apps/

web/

api/

processor/

packages/

world-engine/

rendering/

splat-renderer/

geographic-index/

streaming/

physics/

game-logic/

shared-types/

ui/

infrastructure/

docker/

database/

storage/

cdn/

docs/

architecture/

asset-pipeline/

performance/

development/

Use:

- TypeScript.
- pnpm workspaces.
- Turborepo.
- Strict TypeScript configuration.
- ESLint.
- Prettier.
- Automated tests.

Maintain separation between rendering, geographic systems, game logic and application UI.

React must not manage every frame of the rendering loop.

The 3D engine must have its own lifecycle and rendering state.

---

16. DEVELOPMENT EXECUTION PLAN

DO NOT attempt to implement every feature simultaneously.

Work in phases.

PHASE 0 — Repository inspection and planning

- Inspect the existing repository.
- Identify existing architecture.
- Check existing dependencies.
- Identify reusable code.
- Research technical options.
- Produce technology decisions.
- Create an implementation plan.

Do not destroy existing working functionality.

PHASE 1 — Real 3D prototype

THIS IS THE FIRST DEVELOPMENT MILESTONE.

Build:

- Browser-based 3D scene.
- Actual reconstructed real-world location.
- Gaussian Splat rendering.
- First-person camera.
- WASD movement.
- Mouse look.
- Touch controls.
- Collision geometry.
- Basic geographic positioning.
- WebGPU/WebGL2 capability handling.
- Performance monitor.

Use a small, legally usable reconstruction dataset.

If no real dataset is available, identify the exact missing asset and provide a documented ingestion workflow.

Do not substitute an unrelated procedural city and claim the milestone is complete.

Acceptance criteria:

- The player can navigate a genuinely reconstructed 3D environment.
- The environment is not a panorama.
- Camera movement produces real spatial parallax.
- Collision works.
- The browser does not load the entire world.
- The scene works on desktop.
- A mobile-compatible mode works.
- Performance is measured and documented.

PHASE 2 — World streaming

Build:

- Geographic chunk manager.
- Asset manifest system.
- Progressive loading.
- LOD.
- Cache.
- Memory management.
- Request cancellation.
- Background decoding.

Acceptance criteria:

- Multiple nearby chunks can load.
- Old chunks can unload.
- Movement does not require a full-page reload.
- Performance remains stable during navigation.

PHASE 3 — Geographic foundation

Build:

- Coordinate system.
- Terrain integration.
- Road and building data.
- Geographic index.
- Location coverage database.
- Quality validation.

PHASE 4 — Single-player game

Build:

- Random playable location selection.
- Guess map.
- Guess submission.
- Geodesic scoring.
- Results screen.
- Basic game modes.

PHASE 5 — Living environment

Build:

- Lightweight pedestrian animation.
- Vehicle movement.
- Weather effects.
- Dynamic lighting.
- Environmental simulation.

PHASE 6 — Multiplayer

Build:

- Room creation.
- Matchmaking.

- Player synchronization.
- Round synchronization.
- Competitive scoring.
- Leaderboards.

---

17. CODE QUALITY REQUIREMENTS

The codebase must be maintainable.

Strict rules:

- No giant source files containing unrelated functionality.
- No duplicated implementations.
- No arbitrary dependencies.
- No fake API integrations.
- No placeholder buttons presented as working features.
- No hardcoded secrets.
- No swallowed errors.
- No unnecessary console logging.
- No untyped API responses.
- No unsafe global state.
- No unnecessary re-rendering.
- No blocking main-thread processing for expensive asset decoding.
- No silent failures when an asset cannot load.

Use:

- Modular components.
- Typed interfaces.
- Error boundaries.
- Structured logging.
- Proper resource disposal.
- Loading and error states.
- Unit tests.
- Integration tests.
- Browser performance tests.

Do not claim a feature is complete unless it actually works.

If a feature depends on unavailable external services, explain the dependency and build the correct interface around it.

---

18. TESTING AND VALIDATION

Create a repeatable testing procedure.

Test:

1. Integrated GPU laptop.
2. Dedicated GPU desktop.
3. Mid-range Android phone.
4. iPhone.
5. Slow internet.
6. Network interruption.
7. Browser refresh.
8. Asset download failure.
9. Memory pressure.
10. WebGPU unavailable.
11. WebGL2 fallback.
12. Long exploration sessions.

Measure:

- Average FPS.
- Frame-time percentiles.
- Initial loading time.
- Chunk loading time.
- Downloaded data.
- Memory usage where measurable.
- GPU resource behavior.
- Number of loaded chunks.
- Cache eviction behavior.
- Failed asset requests.

Create a performance dashboard during development.

Do not optimize based solely on subjective visual impressions.

---

19. DOCUMENTATION

Maintain:

docs/architecture/OVERVIEW.md

docs/architecture/TECHNOLOGY_DECISIONS.md

docs/architecture/WORLD_STREAMING.md

docs/architecture/COORDINATE_SYSTEM.md

docs/asset-pipeline/RECONSTRUCTION.md

docs/asset-pipeline/PUBLISHING.md

docs/performance/BENCHMARKS.md

docs/development/SETUP.md

docs/development/ROADMAP.md

Keep documentation updated as implementation changes.

---

20. HOW YOU MUST WORK AS AN AI CODING AGENT

Follow this working protocol.

1. Inspect the repository before editing anything.
2. Research uncertain technical decisions.
3. Explain the chosen architecture.
4. Identify blockers and external dependencies.
5. Create the project structure.
6. Implement the first complete working vertical slice.
7. Run the application.
8. Test it in an actual browser.
9. Fix errors.
10. Measure performance.
11. Update documentation.
12. Report exactly what works and what remains incomplete.

Do not spend the entire task writing documentation or planning.

You must produce working software.

However, do not pretend a missing reconstruction dataset, rendering capability or external service exists.

If the environment lacks an essential asset, state that clearly and implement the ingestion path needed to supply it.

Do not build fake functionality to make the application look complete.

Do not stop after creating a landing page.

Do not replace the actual 3D engine with a visual mockup.

Do not make arbitrary architectural changes without explaining their consequences.

When the project is too large to finish in one session, complete the current milestone properly and leave the repository in a working, documented state.

FINAL INSTRUCTION

Your first objective is not to build a GeoGuessr interface.

Your first objective is to prove that a photorealistic, genuinely navigable, real-world 3D environment can run smoothly inside a browser on average consumer hardware, including mobile devices.

Build the rendering foundation correctly.

Build the streaming system correctly.

Build the geographic data pipeline correctly.

Then build the game around those systems.

The long-term vision is an explorable, geographically accurate, photorealistic digital Earth that can continuously expand in coverage without requiring users to download or render the entire planet.

Treat Earth Uncovered as a serious graphics-engineering and geospatial software project, not a quick prototype or a generic AI-generated website.

Begin by inspecting the repository and researching the technical options. Then implement Phase 1.